中南林业科技大学

编译原理实验报告

课程名称：	编译原理
专业班级：	软工 3 班
姓　　名：	李佳鹏
学　　号：	20245428

完成日期：2026 年 11 月

目录

实验三　LR（1）分析法.....1
一、　实验目的.....1
二、　实验内容.....1
三、　实验理论依据.....6
四、　测试数据及运行结果展示.....9

实验三　LR（1）分析法

一、 实验目的

根据预先由文法构造的 LR（1）分析表，编制一个自底向上的语法分析程序，使其能够对任意输入的符号串进行分析，判断该串是否为给定文法的句子，并把分析过程中状态栈、符号栈、剩余输入串与所执行动作的变化逐行输出。

通过本次实验，理解 LR 分析法“把判断前移到表的构造阶段”这一设计思想：运行期的分析程序只做查表与栈操作，何时移进、何时按哪条产生式归约，全部由 ACTION 表与 GOTO 表预先确定；并掌握 LR（1）项目、向前看符号、项目集规范族与状态转移函数这些构造分析表的基础概念。

二、 实验内容

（1）给出待分析的文法。文法共 4 条产生式，见表 1。其中 E、S、B 为非终结符，a、b 为终结符，# 用作输入串的结束符。第 (0) 条是为构造分析表而引入的增广产生式。

表 1　LR（1）分析所用文法的产生式

序号	产生式	说　明
（0）	E → S	增广产生式，把开始符号 S 提升为 E，便于判定接受
（1）	S → BB	句子由两个 B 相接而成
（2）	B → aB	在 B 的前面读入一个 a
（3）	B → b	B 以 b 结束，a 的个数可以为 0

由这 4 条产生式可以看出，该文法描述的语言是 $L(G)=\{a^n b\, a^m b \mid n \geqslant 0, m \geqslant 0\}$，即两个以 b 结尾的 a 串首尾相接。例如 bb、abab、aabb、aabab 都是句子，而 b、ab、a、baba 都不是。这个语言无法用 LL（1）文法描述，因为两个 B 的分界必须读到第二个 b 才能确定，只能靠自底向上的归约来解决。

（2）分析表的存储。分析表由实验指导书按文法预先构造，程序运行期不再重新构造，直接以两个二维数组的形式硬编码在源程序中：ACTION 表 10 行 3 列，行对应状态 0～9，列依次对应终结符 a、b、#；GOTO 表 10 行 2 列，列依次对应非终结符 S、B。表项用字符串表示，第一个字符区分动作类型，后面的数字是状态号或产生式号。

```
/* ACTION 表：行是状态 0~9，列依次是终结符 a、b、#      */
const char *action[10][3] = {
    {"S3","S4",NULL},   {NULL,NULL,"acc"},  {"S6","S7",NULL},
    {"S3","S4",NULL},   {"r3","r3",NULL},   {NULL,NULL,"r1"},
    {"S6","S7",NULL},   {NULL,NULL,"r3"},   {"r2","r2",NULL},
    {NULL,NULL,"r2"}
};

/* GOTO 表：行是状态 0~9，列依次是非终结符 S、B        */
int goto1[10][2] = {
    {1,2},   {0,0},   {0,5},   {0,8},   {0,0},
    {0,0},   {0,9},   {0,0},   {0,0},   {0,0}
};
```

表项的约定是：S 加数字表示移进，把该状态压栈；r 加数字表示按第几号产生式归约；acc 表示接受；NULL 表示该格为空，一旦查到此格就说明输入串有语法错误。GOTO 表中的 0 只起占位作用，归约后新栈顶状态遇该非终结符并无转移，实际分析时不会查到。

（3）栈、输入串与分析程序的初始化。文法符号分成终结符与非终结符两组存放，查表时用来定位行号与列号；产生式用结构体 PROD 表示，右部长度 len 就是归约时需要弹出的符号个数，左部字符则用来在归约后查 GOTO 表。

```
char vt[3] = {'a', 'b', '#'};     /* 终结符 */
char vn[2] = {'S', 'B'};          /* 非终结符 */

struct PROD {                     /* 一条产生式 */
    char left;                    /* 左部非终结符 */
    char right[4];                /* 右部符号串 */
    int  len;                     /* 右部长度 */
};
PROD P[4] = {
    {'E', "S",  1},               /* 0: E -> S  */
    {'S', "BB", 2},               /* 1: S -> BB */
    {'B', "aB", 2},               /* 2: B -> aB */
    {'B', "b",  1}                /* 3: B -> b  */
};
```

状态栈用整型数组存放状态号，符号栈用字符数组存放文法符号，两栈大小相同、同步变化，栈顶指针共用一个 top；输入串连同结束符 # 一起存入字符数组，ip 指向当前待处理的符号：

```
int  S[80];                    /* 状态栈，栈底 S[0] = 0 */
char X[80];                    /* 符号栈，栈底 X[0] = '#' */
int  top;                      /* 两栈共用的栈顶指针 */
char in[80];                   /* 输入串，以 # 结尾 */
int  ip;                       /* 当前输入符号下标 */
int  step;                     /* 步骤号 */
```

分析开始前，状态栈压入初态 0，符号栈压入结束符 #，输入指针 ip 指向输入串的第一个符号，步骤号置 1。这三个栈底元素是后面所有动作的基准，任何一步归约弹栈都不能把它们弹出。

```
top = 0;  S[0] = 0;  X[0] = '#';  ip = 0;  step = 1;
```

（4）主控程序。每一轮先取栈顶状态与当前输入符号查 ACTION 表，再按表项的类型分四路处理。查表之前要先把输入符号映射成列号、把状态直接作为行号，两个查表函数用最朴素的循环实现：

```
int colOf(char c)              /* 终结符 c 的列号 */
{
    for (int i = 0; i < 3; i++)
        if (vt[i] == c) return i;
    return -1;                 /* 不属于终结符集合 */
}

int rowOf(char c)              /* 非终结符 c 的列号 */
{
    for (int i = 0; i < 2; i++)
        if (vn[i] == c) return i;
    return -1;
}
```

主控循环每轮先取栈顶状态与当前输入符号查表，再按表项分四路处理。表项为空则报错，表项为 acc 则接受，这两种情况都立即结束分析：

```
while (1) {
    int   st  = S[top];                              /* 栈顶状态 */
    int   col = colOf(in[ip]);                        /* 当前输入符号的列 */
    const char *act = (col >= 0) ? action[st][col] : NULL;

    if (act == NULL) { show("error", ""); return; }  /* 表项为空 */
    if (act[0] == 'a') { show("acc", ""); return; }  /* 接受 */
```

表项非空且不是 acc 时，按首字符区分移进与归约：首字符为 S 是移进，把当前输入符号与目标状态分别压入符号栈、状态栈，输入指针后移；首字符为 r 是归约，先按产生式右部长度弹栈，再把左部非终结符与新状态压栈：

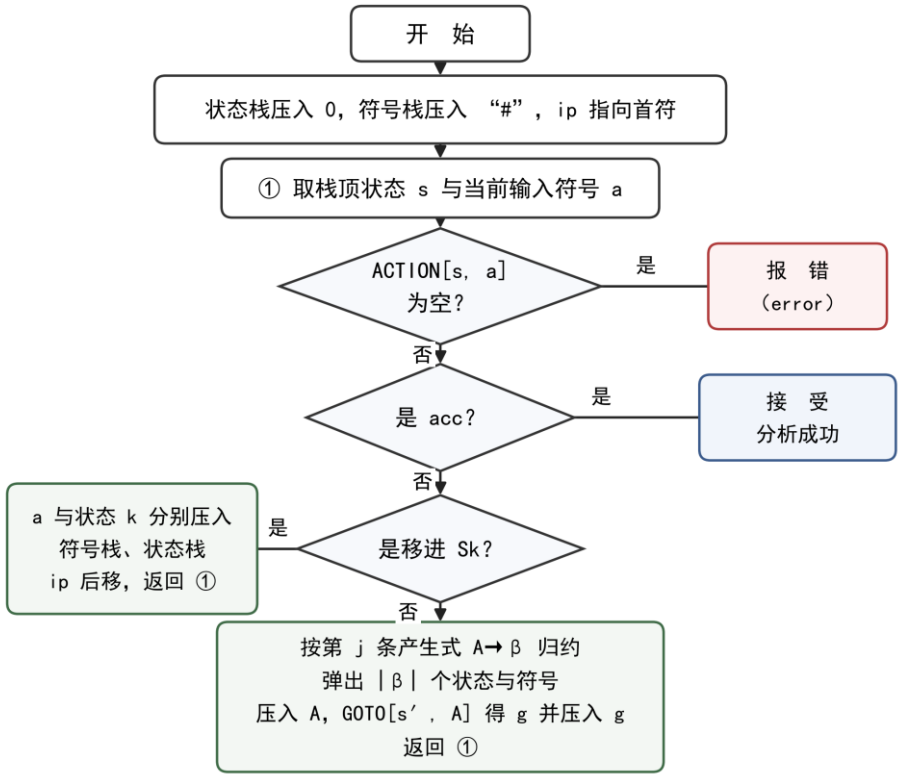
```
    if (act[0] == 'S') {                  /* 移进 */
        show(act, "");
        X[++top] = in[ip];                /* 当前符号入符号栈 */
        S[top]   = atoi(act + 1);         /* 目标状态入状态栈 */
        ip++;  step++;
    } else {                             /* 归约 */
        int  k = atoi(act + 1);           /* 所用产生式号 */
        int  g = goto1[S[top - P[k].len]][rowOf(P[k].left)];
        char buf[8];  sprintf(buf, "%d", g);
        show(act, buf);                   /* GOTO 列显示目标状态 */
        top -= P[k].len;                  /* 弹出右部长度个符号 */
        X[++top] = P[k].left;             /* 左部非终结符入栈 */
        S[top]   = g;                     /* 新状态入栈 */
        step++;
    }
    /* 分支结束，回到 while 开头进入下一轮 */
```

归约这一支有两处容易写错。一是弹栈的个数：要弹出产生式右部的长度 len，而不是固定弹一个，否则栈里的状态与已识别的符号串就不再对应；二是查 GOTO 的时机：必须在弹栈之后、用新的栈顶状态去查，程序中写成 S[top - P[k].len] 正是为了在弹栈之前就取出归约后的栈顶状态。

（5）分析过程的输出。输出函数把当前的状态栈、符号栈、剩余输入串与本次动作拼成一行，各列用 setw 定宽对齐，GOTO 一列只在归约时才有值：

```
void show(const char *act, const char *go)
{
    char a[16], b[16];
    stateStr(a);                        /* 状态栈拼成字符串 */
    symStr(b);                          /* 符号栈拼成字符串 */
    cout << setw(4)  << left << step
         << setw(9)  << left << a
         << setw(9)  << left << b
         << setw(12) << left << (in + ip)
         << setw(8)  << left << act
         << setw(4)  << left << go << "\n";
}
```

每输出一行，打印的都是“本次动作执行之前”的格局，因此第 1 行总是 0 / # / 整个输入串，最后一行总是 acc 或 error。整个分析程序的控制流程如图 1 所示。



注：① 为循环入口；移进与归约两个分支执行后都回到 ①，直到 acc 或 error。

图 1 LR（1）分析总控程序流程图

（6）错误处理。程序把“表项为空”作为唯一的出错判据，它涵盖了两种情况：一是当前输入符号不属于终结符集合（如输入了非法字符），查表时列号取不到；二是符号本身合法，但当前状态不允许读入它，ACTION 表中该格为空。两种情形都立即输出 error 并终止分析，不做错误恢复，因此对于含多处错误的输入串只能报出第一处。

三、 实验理论依据

1．LR 分析法的基本思想

LR 分析法是一种自底向上的语法分析方法。它从输入串开始，一边把符号移进栈，一边在栈顶形成句柄时把句柄归约为产生式左部的非终结符，直到栈中只剩开始符号且输入串读完为止。这个过程恰好是最右推导的逆过程，所以每一步归约得到的都是规范句型，称为规范归约。

LR 分析器由四部分组成：一个存放文法符号与状态的分析栈、一个输入缓冲区、一张分析表和一个总控程序。分析栈中交替存放状态与符号，栈顶状态概括了从分析开始到当前为止已移进和归约出的全部信息，这是 LR 分析不需要回溯的根本原因。由于状态数有限，LR 分析法实际上是用一个有穷自动机来识别文法句子的活前缀，它能处理的文法类是所有能用确定下推自动机识别的文法，覆盖面远大于 LL（1）文法。

2．LR（1）项目与项目集规范族

LR 分析需要知道“在当前的产生式右部中已经识别到哪个位置”，这用一个圆点表示，称为项目。LR（1）项目在此基础上再带一个向前看符号，记作 [A → α • β, a]，含义是：已经识别出 α，期望继续看到能从 β 推出的串，并且 β 之后紧跟的终结符是 a。向前看符号的作用是缩小归约的适用范围，只有当实际输入符号等于 a 时才允许按 A → α β 归约。

从增广产生式 E → •S 出发，反复求闭包和转移，得到该文法的项目集规范族，共 10 个状态，见表 2。求闭包时，若 [A → α •Bβ, a] 在项目集中，且 B 的产生式有 B → γ，则把 [B → • γ, b] 加入，其中 b 取 FIRST(βa) 中的终结符。

表 2　LR（1）项目集规范族

状态	项目集（圆点位置与向前看符号）	转移来源
0	E→ • S,#　S→ • BB,#　B→ • aB,a/b　B→ • b,a/b	初态
1	E→S • ,#	GO(0, S)
2	S→B • B,#　B→ • aB,#　B→ • b,#	GO(0, B)
3	B→a • B,a/b　B→ • aB,a/b　B→ • b,a/b	GO(0, a)
4	B→b • ,a/b	GO(0, b)
5	S→BB • ,#	GO(2, B)
6	B→a • B,#　B→ • aB,#　B→ • b,#	GO(2, a)
7	B→b • ,#	GO(2, b)
8	B→aB • ,a/b	GO(3, B)
9	B→aB • ,#	GO(6, B)

表 2 中有两组状态的项目核心相同而向前看符号不同：状态 3 与状态 6 的核心都是 B → a • B，但状态 3 是从初态出发、正在分析第一个 B，其向前看符号是 a 或 b（第一个 B 后面必跟第二个 B 的首符），状态 6 是从状态 2 出发、正在分析第二个 B，其向前看符号只有 #。状态 8 与状态 9 同理。这正是 LR（1）与 LR（0）、SLR 的关键差别：项目相同但上下文不同，就不能合并。

3. GO 函数与状态转移

GO(I, X) 表示从项目集 I 出发、越过文法符号 X 后得到的项目集，即把 I 中所有圆点紧挨在 X 左边的项目去掉 X、把圆点右移一位，再求闭包。全部转移关系见表 3，其中“—”表示没有转移。

表 3　状态转移函数 GO

状态	遇 a	遇 b	遇 S	遇 B
0	3	4	1	2
1	—	—	—	—
2	6	7	—	5
3	3	4	—	8
4	—	—	—	—
5	—	—	—	—
6	6	7	—	9
7	—	—	—	—
8	—	—	—	—
9	—	—	—	—

表 3 中“遇 a”与“遇 b”两列对应 ACTION 表中的移进项，“遇 S”与“遇 B”两列对应 GOTO 表。可以看出状态 3 与状态 6 在遇 a、b 时的转移目标完全相同，差别只在遇 B 时分别转到 8 和 9，这与表 2 中两组状态向前看符号不同是同一件事的两种表现。

4．ACTION 表与 GOTO 表的构造

分析表按下列四条规则填写。对每个状态 i：若 [A → α •a β，b] ∈ I_i 且 GO(I_i，a) = I_j，其中 a 为终结符，则置 ACTION[i，a] = sj；若 [A → α •，a] ∈ I_i 且 A ≠ E，则置 ACTION[i，a] = rj，j 为该产生式的编号；若 [E → S •，#] ∈ I_i，则置 ACTION[i，#] = acc；若 GO(I_i，A) = I_j，A 为非终结符，则置 GOTO[i，A] = j。

按上述规则填出的表见表 4，它与实验指导书给出的表完全一致。表中空白格表示出错，程序里用 NULL 表示；r1、r2、r3 分别对应 S → BB、B → aB、B → b 三条产生式，0 号产生式只用于产生状态 1，不出现在归约项中。

表 4　LR（1）分析表

状态	ACTION			GOTO	
	a	b	#	S	B
0	S3	S4		1	2
1			acc		
2	S6	S7			5
3	S3	S4			8
4	r3	r3			
5			r1		
6	S6	S7			9
7			r3		
8	r2	r2			
9			r2		

5．分析格局与四种动作

分析过程中任一时刻的格局记作 ($s_0X_1s_1X_2\cdots X_ms_m$，$a_ia_{i+1}\cdots a_n$#)，前半部分是栈的内容（自底向顶），后半部分是尚未处理的输入串。设栈顶状态为 s_m、当前输入符号为 a_i，则按 ACTION[s_m，a_i] 的取值执行四种动作之一。

移进：若 ACTION[s_m，a_i] = s，则把 a_i 与状态 s 依次压栈，格局变为 ($\cdots a_is$，$a_{i+1}\cdots a_n$#)。归约：若 ACTION[s_m，a_i] = rj，第 j 条产生式为 A → β，|β| = r，则从栈顶弹出 r 个符号与 r 个状态，设此时栈顶状态为 s′，再把 A 与 GOTO[s′，A] 压栈，格局变为 ($\cdots As$，$a_i\cdots a_n$#)，输入指针不动。接受：若 ACTION[s_m，a_i] = acc，分析成功。报错：若表项为空，说明输入串不是文法的句子。

这四种动作里，归约是唯一会让栈变短的动作，也是最容易与“弹出几个”混淆的地方。由于栈中状态与符号交替出现，弹出 r 个符号的同时必须弹出 r 个状态，否则状态与符号的配对关系就会错位，后续所有查表都会失效。

6. LR（1）与 SLR 的差别

SLR 方法只用 FOLLOW 集来决定归约，本实验中 FOLLOW(B) = {a, b, #}，若按 SLR 填表，状态 4 与状态 7 在 a、b、# 三列上都应填 r3。但按 LR（1）填表，状态 4 只在 a、b 上填 r3，因为状态 4 中的项目是 [B → b •, a/b]，向前看符号不含 #。

这一差别在运行结果中可以直接看到：输入串 b# 只有两个符号，程序在第 2 步、即状态 4 遇到 # 时就报错；若按 SLR 填表，状态 4 遇 # 会先归约成 B，再在状态 2 遇 # 才报错，要多走一步。可见向前看符号越精确，错误发现得越早，这也是 LR（1）比 SLR 更强的原因。

四、 测试数据及运行结果展示

按表 1 的文法，合法句子应具有 $a^n b\ a^m b$ 的形式。共设计 8 组测试数据，既有合法串也有非法串，覆盖 a 的个数为 0、1、2 以及两个 B 长度不等的各种情况，汇总见表 5。

表 5　8 组测试数据及运行结果汇总

序号	输入串	是否句子	步数	终止动作	说　明
1	bb#	是	6	acc	n = 0、m = 0，最短的句子
2	baba#	否	5	error	两个 B 之后多出一个 a
3	abab#	是	10	acc	n = 1、m = 1，两 B 均为 ab
4	aabab#	是	12	acc	n = 2、m = 1，两 B 不等长
5	aabb#	是	10	acc	n = 2、m = 0，首个 B 为 aab
6	ab#	否	3	error	缺少第二个 B，状态 4 报错
7	a#	否	2	error	只有 a，没有 b 结尾
8	b#	否	2	error	只有一个 B，缺少另一个

8 组数据中 4 个合法串全部被接受，4 个非法串全部被拒绝，且报错位置与手工推导一致。下面给出 bb#、baba# 与 abab# 三组数据的完整分析过程，其余各组的输出格式相同。

（1）输入串 bb#。该串由两个最短的 B 相接，共 6 步：先移进 b 归约为 B，再移进 b 归约为 B，最后用 S → BB 归约，栈中只剩 #S 时接受。分析过程见表 6。

表 6　输入串 bb# 的分析过程

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	#	bb#	S4	
2	04	#b	b#	r3	2
3	02	#B	b#	S7	
4	027	#Bb	#	r3	5
5	025	#BB	#	r1	1
6	01	#S	#	acc	

表 6 的第 2 步是归约,状态栈由 04 变为 02、符号栈由 #b 变为 #B,GOTO 列给出 2，说明归约后栈顶状态是 2；第 5 步用 S → BB 归约，一次弹出两个 B 与两个状态，状态栈由 025 变为 01。归约行的 GOTO 值都能与表 3 的转移关系对应上。

（2）输入串 baba#。该串在第 5 步出错，是实验指导书给出的错误示例。前 4 步依次移进 b、归约为 B、移进 a、移进 b，此时栈中已有 #Bab、输入还剩 a#，而状态 7 的项目是 [B → b •, #]，只允许在 # 上归约，遇 a 无动作可查，故报错。分析过程见表 7。

表 7　输入串 baba# 的分析过程

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	#	baba#	S4	
2	04	#b	aba#	r3	2
3	02	#B	aba#	S6	
4	026	#Ba	ba#	S7	
5	0267	#Bab	a#	error	

从表 7 可以看出，出错时栈中的符号串是 #Bab，其中第二个 B 已经读到 ab，但还没有归约；此时又出现一个 a，意味着句子中出现了第三个 B，而 S → BB 只允许两个 B，因此该串不是句子。程序在第 5 步直接报错，没有做任何恢复，这与实验设计的“遇错即停”策略一致。

（3）输入串 abab#。两个 B 都是 ab，分析共 10 步，含两次 B → aB 归约，可验证 a 的读入。分析过程见表 8。

表 8　输入串 abab# 的分析过程

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	#	abab#	S3	
2	03	#a	bab#	S4	
3	034	#ab	ab#	r3	8
4	038	#aB	ab#	r2	2
5	02	#B	ab#	S6	
6	026	#Ba	b#	S7	
7	0267	#Bab	#	r3	9
8	0269	#BaB	#	r2	5
9	025	#BB	#	r1	1
10	01	#S	#	acc	

表 8 中第 3、4 步把 ab 归约成 B：先用 B → b 把栈顶的 b 归约，GOTO 得状态 8，再用 B → aB 把 aB 归约成 B，GOTO 得状态 2。第 7、8 步对第二个 ab 重复同样的过程，但 B → b 归约后进入的状态由 8 变成了 9，这正是表 2 中状态 8 与状态 9 向前看符号不同的体现。

综合 8 组数据可以看出：分析步数与输入串长度大致成正比，每个符号至少要移进一次，每个 B 至少归约一次；两个 B 的分界点必须读完第二个 b 才能确定，这也解释了为什么该语言不能用 LL（1）方法分析。

结　论

本次实验完成了 LR（1）分析法的全部实践环节：以 4 条产生式为基础，构造出含 10 个状态的项目集规范族，填出 10 行 3 列的 ACTION 表与 10 行 2 列的 GOTO 表，并用 C++ 实现了表驱动的分析程序。8 组测试数据判断全部正确，4 个合法串分别在 6、10、12、10 步后接受，4 个非法串分别在 5、3、2、2 步报错，状态栈与输入串的变化与手工推导逐行一致。

实验中的主要难点有三处。一是归约时弹栈的个数：要按产生式右部长度弹出同样多的状态与符号，只弹一个就会使配对错位；二是 GOTO 的查表时机：必须用归约后的新栈顶状态去查；三是向前看符号的作用：它把归约限定在特定的输入符号上，状态 4 只对 a、b 归约而不对 # 归约。

通过这次实验，我最深的体会是：LR 分析把“何时移进、何时归约”的判断前移到分析表的构造阶段，运行期的程序只剩查表与栈操作，结构简单而能处理的文法更广。代价是表的构造复杂：项目集、闭包、向前看符号算错一处，表就会多出或少掉表项，错误往往要到特定输入串上才暴露。